

Activity 1: Requirements and Component Boundary

Course: AI Vibe Coding for iOS Mobile Apps Development (TGS-2023037545)

Mapping: K1, A1

Estimated time: 70 minutes

Objective

Convert stakeholder needs into testable requirements and design the SwiftUI–Objective-C++–C++ boundary.

Deliverable

Requirement matrix, boundary diagram and acceptance criteria

Tools

Markdown, diagrams.net or PowerPoint, JSON fixture

Included files

- ``starter/scenario.txt``
- ``starter/requirements-template.txt``
- ``solution/requirements-matrix.txt``
- ``mock-data/user-stories.json``
- ``solution/boundary-diagram.txt``

Before you begin

- Work only with the supplied mock data. Do not add credentials, personal data or signing files.
- Keep the portable C++ core free of Swift, Objective-C and Apple framework dependencies.
- Record the initial state before editing so that verification shows a meaningful change.
- If using an AI coding assistant, treat its output as a proposal and inspect the resulting diff.

Procedure

1. Open a terminal at the repository root and run ``cd activities/01-requirements-component-boundary`` followed by ``find . -maxdepth 3 -type f | sort``. Confirm that the scenario, template, solution example and mock user stories are present.
2. Read the brief with ``sed -n '1,160p' starter/scenario.txt`` and inspect the structured stories with ``python3 -m json.tool mock-data/user-stories.json``. Record the two actors and their intended outcomes before proposing components.
3. Copy ``starter/requirements-template.txt`` to a working file named ``requirements-working.txt``. Do not edit the supplied starter or solution reference.

4. Add five unique requirement IDs `REQ-001` through `REQ-005`. For every row, complete actor, input, business rule, observable outcome and acceptance evidence; avoid implementation verbs such as click or instantiate.
5. For the money rules, state the unit explicitly as signed 64-bit integer cents. Add acceptance examples for a valid expense, zero amount, negative amount and an expense above the remaining budget.
6. Create `boundary-working.txt` and write the dependency chain `SwiftUI View -> BudgetViewModel -> BudgetBridge.h -> BudgetBridge.mm -> ExpenseService -> ExpenseRepository`.
7. Under the chain, assign one responsibility to every layer. State that `DomainCore` imports no Swift, Objective-C, UIKit, SwiftUI or Foundation type.
8. For each requirement, name the domain operation that owns the business verb, such as `BudgetPolicy::evaluate`, rather than naming a screen widget.
9. Review the completed matrix against `solution/requirements-matrix.txt` and the boundary against `solution/boundary-diagram.txt`. Differences are acceptable only when they preserve the regulated A1 design intent.
10. Run `rg -n 'REQ-[0-9]{3}|SwiftUI|Objective-C|C|C++' requirements-working.txt boundary-working.txt`. Expect all five IDs and all three technology boundaries to be visible.
11. Create `acceptance-review.txt` and record PASS or REVISE for uniqueness, testability, explicit units, observable outcomes and inward dependency direction.
12. Verification: a reviewer must be able to select any `REQ-\*` row and point to exactly one interface operation plus one reproducible acceptance observation. Retain the three working text files as Activity evidence.

Verification

Every requirement links to one interface operation and one observable acceptance criterion.

Capture the command output, test result, matrix or screenshot named by the deliverable. A statement such as “it works” is not sufficient evidence.

Troubleshooting

If the boundary follows screen widgets, restate each operation as a business verb before redrawing.

Check the earliest failing layer first: portable C++ domain test, then Objective-C++ contract, then Swift state and finally the visible interface.

Cleanup

Retain the versioned design artifacts; remove only disposable sketches.

Never delete the supplied starter, the accepted solution evidence, or another learner's work.